



牛客竞赛

AC.NOWCODER.COM

2023 牛客 暑期多校训练营

第二场 黑冰茶命题组

HR.NOWCODER.COM





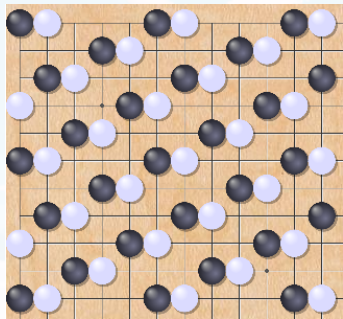
I - Link with Gomoku

题目大意

- 在 $n \times m$ 的棋盘上，构造一个五子棋的和棋局面
- $n, m \leq 10^3$

- 签到题。
- 可能的 Wrong Answer 原因包括：
 - 黑子数量不是 $\lceil \frac{n*m}{2} \rceil$
 - 白子数量不是 $\lfloor \frac{n*m}{2} \rfloor$
 - 存在横、竖、两个对角线方向的五子连珠

- 一种构造方法如右图所示
- 容易发现，双方均不可能形成五子连珠了
- 多余的棋子随便放置即可



F - Link with Chess Game

题目大意

- 有一条长为 n 的链，红绿蓝三颗棋子
- 双方轮流移动一颗棋子，三颗棋子位置构成的有序三元组不能重复
- 不能移动者输，问谁赢
- $n \leq 10^5$

K

ordered triple 翻译成中文应该是什么意思



"ordered triple" 的中文翻译是 "有序三元组"。

在数学和计算机科学中，ordered triple 指的是一个由三个元素按照特定顺序组成的有序组合。通常表示为 (a, b, c) ，其中 a 、 b 和 c 是三个元素，顺序是有意义的，即 (a, b, c) 和 (b, a, c) 是不同的 ordered triple。这种概念在向量、坐标和组合数学等领域经常使用。

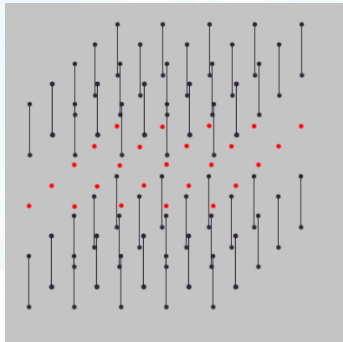
题解

- 本题是一道诈骗题/猜结论题/分类讨论题
- 对于“不能经过重复状态”的博弈问题，通常首先考虑其是否为二分图，若是，则可以使用二分图博弈算法
- 在本题中，状态转移构成的图是一个边长为 n 的三维立方体，因此显然是二分图
- 二分图博弈结论：若起始状态必定位于该二分图的最大匹配上，则先手必胜。否则先手必败。
- 但本题 $n = 10^5$ ，无法用二分图博弈的一般解法求解

- 当 n 是偶数时，最大匹配一定是满的，先手必胜
- 当 n 是奇数时，容易得到只多一个点的匹配
 - 若起点在二分图较少的一部，先手必胜
 - 若起点在二分图较多的一部，可以证明任何一个点都不必然在最大匹配上，后手必胜

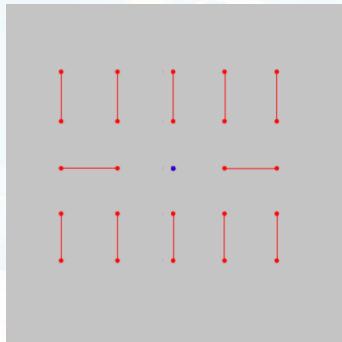
以下证明这个结论：

- 对二分图较多一部中的点 (r_0, g_0, b_0) ，有 $r_0 + g_0 + b_0$ 是奇数
- r_0, g_0, b_0 中必有至少一个奇数，不妨设 r_0 是奇数，构造右图的匹配方案以去掉 r 这一维



题解

- 考虑 g_0, b_0 。若 g_0, b_0 均为奇数，按右图方法构造。
- 否则，用同样方法将除以 (r_0, b_0) 为中心的 3×3 网格以外的部分匹配满。对于 3×3 的网格，显然存在不包含中心点的匹配 4 对点的方案。



花絮

- 1 题目中的 $\sum n$ 是用来误导你的。甚至有验题人要求将 n 的范围改成 100，让它看起来更像是某种神秘复杂度的做法。
- 2 有验题人用一堆分类讨论通过了本题，想必正式赛上肯定也有这么过的。
- 3 这题对棋子的数量应该没有要求，应该都能构造出满的匹配。



E - Square

题目大意

- 多组询问，给定整数 x ，问是否存在整数 $0 \leq y \leq 10^9$ 使得 y^2 在十进制下以 x 开头。
- $0 \leq x \leq 10^9$



题解

- 枚举 y^2 的位数，开根判断即可。注意精度。

K - Box

题目大意

- 给出一个长度为 n 的 01 串以及每个位置都有权值 $a_i \geq 0$ 。
- 每个 1 最多移动一次且最多移动一位。
- 若一个位置有 1 则可以获得权值。
- 问最大可能的权值和。
- $n \leq 10^6$ 。

- $f_{i,0/1,0/1/2}$ 表示前 i 个位置，当前位置是否已经盖有 1，下一个位置是否也盖有 1，有的话是否从当前位置移动过去的最大权值和，转移讨论即可。
- 时间复杂度 $O(n)$ 。

H - 0 and 1 in BIT

题目大意

- 给定一个长为 n 的只含 A, B 两种字符的字符串
- 给定 Q 次询问 (l, r, x) 表示二进制字符串 x 经过字符串 (l, r) 这段区间后变成什么
- 之中, A 操作反转该二进制字符串 (0, 1 互换), B 操作将该二进制字符串视为数字计算 $x = x + 1$ (溢出的位舍弃)
- 数据范围都在 2×10^5 级别, 且询问要求强制在线

题解

- 首先一个很关键的观察是，操作一可以等同于：
- $x = -x - 1$
- 这个观察可以通过两种方式得到：
- 1、我会会计组！由于操作一其实是求反码，而我们知道一个数的反码加一等于补码（补码是 $-x$ ），所以反码是 $-x - 1$
- 2、我会取模！由于操作二可以看作 $x = (x + 1) \% 2^k$ ，之中 k 为二进制串 x 的长度，操作一则是 $x \equiv (2^k - 1 - x) \equiv -x - 1 \pmod{2^k}$

题解

- 根据这个转化，我们发现，可以考虑每个字符对于最终答案的影响，例如：
- BABA 中第一个 B 对答案的影响是 $+1$ （后面有两个 A），第一个 A 对答案的影响是 $+1$ （后面有一个 A），第二个 B 对答案的影响是 -1 （后面有一个 A），第二个 A 对答案的影响是 -1 （后面什么都没了）
- 顺便也能发现对于一段确定的区间 (l, r) 无论输入的 x 是什么，变化都是 x 乘以 1 或 -1 （取决于这段里 A 的奇偶性），再（在模意义下）加或减同一个常数，我们求出每个区间的这个常数就可以得到答案，记这个要求的東西为 $f(l, r)$

题解

- 转化后的问题可以用矩阵加线段树或加倍增或前缀求矩阵逆来维护，不过这里介绍一种十分阳春白雪的前缀和做法
- 记 $\text{cnt}(l, r)$ 表示 (l, r) 中 A 的数量，可以前缀和预处理。同时我们也可以前缀和预处理出前缀的答案 $f(1, 1), f(1, 2), \dots, f(1, n - 1), f(1, n)$
- 对于一次询问 (l, r) ，我们可以用前缀和解出 $f(l, r)$ ：
- 若 $\text{cnt}(l, r)$ 是偶数，则有 $f(1, l - 1) + f(l, r) = f(1, r)$
- 若 $\text{cnt}(l, r)$ 是奇数，则有 $-f(1, l - 1) + f(l, r) = f(1, r)$
- 另外，注意 $\text{cnt}(l, r)$ 是奇数时，要把输入的 x 先乘以 -1 再加 $f(l, r)$

- 本题的强制在线主要是为了防止有人使用 AGC044C 题的做法一下复制黏贴就过了，这也是个很有意思的题
- 其实也确实本题灵感来源，经典之想错的题 == 出了新题
- 本来不强制在线的，验题时 gkjj 拉了这题板子直接秒了，并派出 bbg 嘲笑我出原题，令出题人恼羞成怒，加了强制在线

D - The Game of Eating

题目大意

- 一共有 m 道菜， n 个人轮流点，一共点 k 道。
- 第 i 个人对第 j 道菜的喜爱程度 $A_{i,j}$ 公开，一个人点了菜所有人都可以吃到。
- 每个人都希望最大化自己的喜爱程度之和，求最终的点菜集合。
- $1 \leq n, m \leq 2000, 1 \leq A_{i,j} \leq 10^9, \forall 1 \leq x \neq y \leq m, A_{i,x} \neq A_{i,y}$

一些感性认识：通过样例容易发现，每次贪心地选择自己最喜欢的菜是不优的，因为这道菜也可能成为后面的人的选择，这样我们就浪费了一次机会。考虑如何不浪费这样的机会：

- 假设最后一个人最喜欢的菜在最后还没被选，则最后一个人一定会选它；
- 因此其它人不会浪费机会去选择这道菜；
- 同理，在剩下的菜中倒数第二个人一定会选择他最喜欢的；
- 以此类推，倒过来贪心即可；

AC.

题解

猜到这样贪心就可以 AC 本题，但是正确性的证明比较复杂。下面是由 OMG_link 给出的一个证明：

- 假设所有人都知道，当剩余菜集合为 A 时，最后 k 个人一定会保证 $S[A, k]$ （即我们的策略）集合内的所有菜都被选。进行归纳：
- 1. $k = 1$ 情况显然；
- 2. $k > 1$ 时：

题解

- 记 A 中除 $S[A, k - 1]$ 以外到数第 k 个人最喜欢的菜为 x :
- 2.1 假设第 k 个人选 x , 得到集合 $S[A, k]$
- 2.2 假设第 k 个人选 $S[A, k - 1]$ 和 x 以外的菜, 根据归纳前提, 剩下 $k - 1$ 个菜一定是 $S[A, k - 1]$, 因此第 k 个人一定亏
- 2.3 假设第 k 个人选 $S[A, k - 1]$ 内的菜, 则根据归纳前提, 剩余 $k - 1$ 人中恰有一人选择 $S[A, k - 1]$ 以外的菜, 若:
 - 2.3.1 这个人选的是菜 x , 则得到 $S[A, k]$
 - 2.3.2 这个人选的不是菜 x , 则第 k 个人亏



花絮

本题背景是与 fried-chicken 和 fallleaves 某次吃饭后的心路历程



A - Link with Checksum

题目大意

- 构造一个 checksum , 使得 $\text{CRC-link}(\text{header}, \text{checksum}, \text{footer}) = \text{checksum}$
- CRC-link 即 CRC-32 去掉前后的取反操作
- header 和 footer 的长度 $\leq 10^5$



题解

- CRC 输入的第 i 位翻转, 会使答案异或一个固定的值 f_i
- 这个值可以递推求出

- 设固定输入对 checksum 的贡献为 a ，列出以下方程：

$$(\oplus_{i=0}^{31} b_i f_{i+n_2}) \oplus a = \sum_{i=0}^{31} b_i \cdot 2^i$$

- 方程中各个二进制位独立，可以拆成 32 个方程求解
- 使用高斯消元、线性基等方法均可

花絮

- 1 本题事实上是没有无解情况的，根据出题人对 10^5 以内的 n_2 的枚举，产生的所有方程均是满秩的。但是由于出题人线性代数水平有限，暂时无法给出其必定满秩的证明。若有高手可以证明，欢迎分享给出题组。
- 2 在验题过程中，有验题人提供了折半搜索的算法，似乎是利用了 CRC 算法的某些神秘性质，详情可以看萌新版题解。此种算法复杂度与 CRC 位数相关，使用更多位的 CRC 可以卡掉，但出题人懒为了让更多同学能通过本题，没有卡掉该种做法。

G - Link with Centrally Symmetric Strings

题目大意

- 询问一个串 S 是否由若干“中心对称”的串拼接而成。
- $\sum |S| \leq 5 \times 10^6$ 。

题解

- 参考 Manacher 算法的思路，可以判断任意一个字符为中心，中心对称串的最大半径
- 一个简单的思路：从前往后考虑每个前缀是否是“好的”，对于一个好的前缀，拼接一个中心对称串后，转移到下一个好的前缀。
- 此类思路有众多 $O(n \log n)$ 的做法，但可惜并过不了（也许有人卡过去了）

- 为了在线性时间内解决本题，引出结论：在从前往后转移的过程中，对于每个“好的”前缀，只需要选择其后最短的中心对称串转移即可
- 用这个结论，在 Manacher 的过程中，一旦遇到了覆盖整个前缀的中心对称串，就将这个前缀截掉
- 同 Manacher 算法的证明，此方法复杂度是 $O(n)$ 的

题解

下面证明上文用到的结论：

- 考虑一个中心对称串 $S = AB$ ，其中 A 是该串最短的中心对称前缀
- 首先证明 $|A| \leq \frac{1}{2}|S|$ ：
 - 假设 A 的长度超过了 S 长度的一半，不妨将 S 写做 $B'CB$ ，其中 B 与 B' 中心对称
 - 由于 S 是中心对称串， C 也是中心对称串
 - 由 A 的中心对称性， A 的前缀中含有 C 的中心对称串 C' ，于是 A 不是 S 最短的中心对称前缀

- 于是可以有 $S = ABA'$ ，其中 A, B, A' 都是中心对称串
- 于是可以将“好的”串拆分为许多中心对称串，每个中心对称串的所有前缀都不是中心对称串
- 于是上述转移是正确的

B - Link with Railway Company

题目大意

- 有一个 n 个车站的树形铁路网，每条铁路有维护费用 c_i
- 有 m 条运营线路，每条有收益 x_i 和运营花费 y_i
- 放弃一些铁路的维护，放弃一些运营线路，使收益最大化
- $1 \leq n, m \leq 10^4$

题解

- 最大权闭合子图
- 每条运营线路，树上的每条边各建一个点
- 源点向每条运营线路连边，每条运营线路向运营线路所需的铁路连边，每条铁路向汇点连边。运营线路赋 $x_i - y_i$ 的权值，铁路赋 $-c_i$ 的权值。
- 直接建图会产生至多 nm 条边，可以采用树剖线段树的方式，将边的数量压缩到 $m \log^2 n$ 条
- 用网络流解决最大权闭合子图问题即可



花絮

- 1 本题有 $m \log n$ 的建图方式，但受网络流玄学性能的影响，两种建图方式的运行效率差别不大，某些情况下 $m \log^2 n$ 的建图甚至跑的更快



C - graph

题目大意

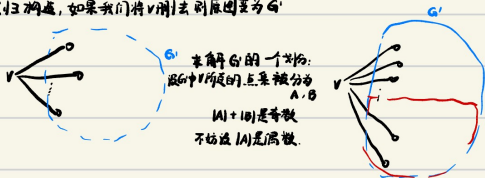
- n 个点 m 条边的无向图
- 将点集分成两个部分，生成对应的导出子图
- 要求两个导出子图中所有连通块都存在欧拉回路，求构造方案
- $1 \leq u_i \leq n \leq 100, 0 \leq m \leq 10^4$

考虑一个更强的构造,使得划分后,两个导出子图的点度数都为偶数:

记原图为 G , ①若 G 的所有点都为偶度点,那么将所有点归入一个划分即可.

②若 $\exists v \in G$ s.t. $\deg(v)$

我们尝试递归构造,如果我们将 v 删去则原图变为 G'



为使 $\deg(w)$ 为偶数,我们必须将 v 归入 A 集合,可是问题出现了: A 中的点都被偶数个点所连接!

为使 $\deg(u)$ 为偶数，我们必须将 v 归入 A 集合，可是问题出现了： A 中的点本来偶数却多了-1条！

为解决这个问题，一个巧妙的构造是：对于原图 G ，设 v 所连的点集为 P ， $|P|$ 为奇数。我们若在删去 v 之后，对 P 的导出子图取补图

得到 $n-1$ 个点的图 G' ，递归求解 G' 的二染色。设 P 中的点被分为两部分， $P = A \cup B$ ， $A \cap B = \emptyset$ 。不妨设 $|A|$ 偶， $|B|$ 奇。

我们必须将 v 归入 A 中（为保证 $\deg(v)$ 是偶数）。

对于 A 中的一个点 u ，在 A 所在的子图中连了 $A_1, A_2, \dots, A_k$
与不在 A 中的 $S_1, S_2, \dots, S_t$ 连边了



$k+t$ 是偶数。那么对于原图 $\deg(u) = |A| - 1 - k + t$ 是奇数，在加入 v 后刚好也是偶数！

J - Smoke

题目大意

- 给出 $f_{0,1}, \dots, f_{0,m}$ 以及递推式 $f_{i,j} = pA_{j-1}f_{i-1,j-1} + (1-p)(B_j + C_i)f_{i-1,j}$ 的各个系数。
- 对每个 $n = 1, 2, \dots, N$ 求 $\sum_j f_{n,j}$ 。
- 对 998244353 取模。
- $1 \leq N, m \leq 10^5$ 。

题解

- 不妨设 n, m 同阶。
- 注意到 $p, 1 - p$ 仅仅是常数，且期望不重要，只要在初始值除以 m 即可。
- 为了接下来书写方便，我们不妨设 $f_{i,j} = A_{j-1}f_{i-1,j-1} + (B_j + C_i)f_{i-1,j}$ 。
- 进一步化简可以发现， $f_{i,j} = A_{j-1}f_{i-1,j-1} + B_jf_{i-1,j} + C_if_{i-1,j}$ ，可以注意到 C_i 的作用仅仅是整体乘上某个定值，并不会影响 $f_{i,j}$ 之间的递推结果。
- 于是如果设 $g_{i,j} = A_{j-1}g_{i-1,j-1} + B_jg_{i-1,j}$ ，以及 $G(x) = \sum_n (\sum_j g_{n,j})x^n$ 的话，此时 $\sum_j f_{n,j} = [x^n]G(x) \prod_{i=1}^n (1 + C_i x)$ 。
- 如果我们能求出 $G(x)$ ，注意到右边的多项式是要乘到 n ，此时我们考虑对分治结构进行维护。做 CDQ 分治。

题解

- 具体而言, 设 $\text{Solve}(l, r, P)$ 表示分治到 $[l, r]$ 时的情况, 考虑中点为 mid 。
- 设 $P' = P_0 \sim P_{\text{mid}-l+1}$, $F = P * \prod_{i=l}^{\text{mid}} (1 + C_i x)$, $P'' = F_{\text{mid}-l+1} \sim F_{r-l+1}$, 其中 $F_x \sim F_y$ 表示多项式 F 第 x 项到第 y 项按序排列起形成的多项式。
- 先递归 $\text{Solve}(l, \text{mid}, P')$, 再递归 $\text{Solve}(\text{mid} + 1, r, P'')$ 。
- 对于 $l = r$ 时, 答案即为 $[x^1]P * (1 + C_i x)$ 。
- 此时的总复杂度是 $O(n \log^2 n)$ 。
- 于是只用考虑 $G(x)$ 如何求得。

题解

■ 考虑 $G_k(x) = \sum_{i=0}^{\infty} g_{i,k} x^i$, 于是有递推式子:

■

$$g_{i,j} x^i = x A_{j-1} g_{i-1,j-1} x^{i-1} + x B_j g_{i-1,j} x^{i-1}$$

$$\Rightarrow \sum_{i=1}^{\infty} g_{i,j} x^i = x A_{j-1} G_{j-1}(x) + x B_j G_j(x)$$

$$\Rightarrow (1 - x B_j) G_j(x) = g_{0,j} + x A_{j-1} G_{j-1}(x)$$

题解



$$G_0(x) = \frac{g_{0,0}}{1 - B_0x}$$
$$\Rightarrow G_k(x) = \sum_{j=0}^k g_{0,j} \frac{1}{h_k(x)} \prod_{i=j}^{k-1} \frac{f_i(x)}{h_i(x)}.$$

■ 其中 $f_i(x) = xA_i$, $h_i(x) = 1 - B_ix$.

题解

■ 于是答案等价于

■

$$\begin{aligned}
 & [x^n] \sum_{k=0}^m \sum_{j=0}^k g_{0,j} \frac{1}{h_k(x)} \prod_{i=j}^{k-1} \frac{f_i(x)}{h_i(x)} \\
 &= [x^n] \sum_{j=0}^m g_{0,j} \sum_{k=j}^m \frac{1}{h_k(x)} \prod_{i=j}^k \frac{f_i(x)}{h_i(x)}
 \end{aligned}$$

题解

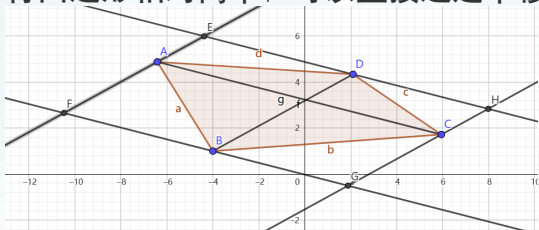
- 可以发现等价于是每个区间都乘起来，同时左右端点处有特殊贡献。
- 考虑分治，每个节点维护一个四元组，分别表示 (区间多项式积的和 A ，固定左端点到区间最右的多项式积的和 L ，固定右端点到区间最左的多项式积的和 R ，区间多项式的积 S)。
- 注意这些多项式本身都是二元组，即有分子和分母。
- 发现可以合并，即设两个多元组为 $(A_l, L_l, R_l, S_l), (A_r, L_r, R_r, S_r)$ ，则可以合并成 $(A_l + A_r + L_l * R_r, L_r + L_l * S_r, R_l + R_r * S_l, S_l * S_r)$ 。
- 时间复杂度仍为 $O(n \log^2 n)$ 。
- 具体实现时，不选择维护二元组而选择统一维护分母并通分是比较合理的，此时多项式的次数将会大幅减少。

L - Link doesn't want to cut tree

题目大意

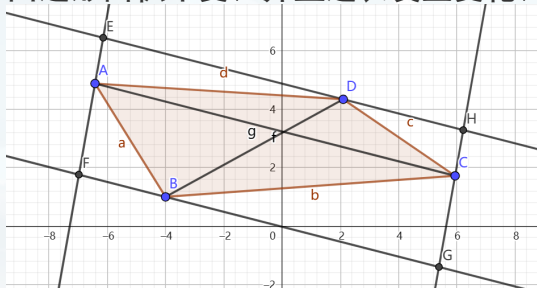
- 多组询问，给定矩形，构造外接菱形使其面积位矩形两倍。
- 需要判断是否无解。

设原矩形面积为 S 。直接构造菱形比较困难，但是构造一个面积为 $2S$ 的平行四边形相对简单，可以直接通过平移对角线完成：

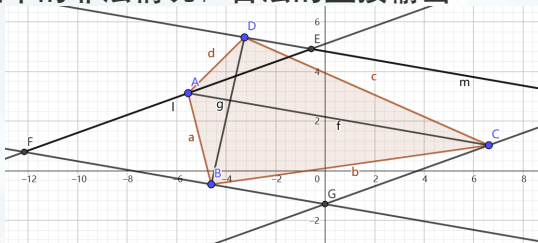


容易证明，平行四边形 EFGH 的面积为 ABCD 的二倍。

接下来分析如何利用 EFGH 来构造结果。发现旋转一条对边可以使平行四边形面积不变，并且边长发生变化，下图为旋转 EF, GH 两条边后的图形：



通过上述旋转操作，计算出 $|EF| = |GH|$ 时对应的旋转角度即可得到 $A'B'C'D'$ 。需要枚举两组对边 *2 个旋转方向共四种情况，并判断是否会出现下图中的非法情况，合法的直接输出：



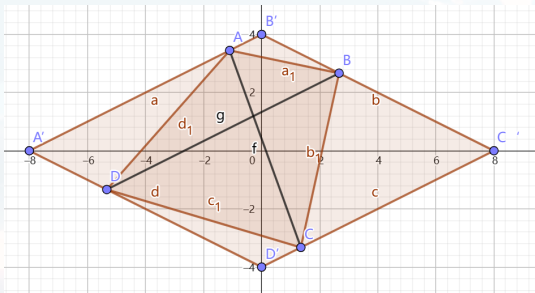
下面说明通过上述方法一定可以构造出解：

- 在 $A'B'C'D'$ 的一组对边与 $ABCD$ 的一条对角线平行时，上述方法都可以构造出结果。
- 那么只要证明 “ $S_{A'B'C'D'} = 2S_{ABCD}$ 时， $A'B'C'D'$ 一组对边一定与 $ABCD$ 的一条对角线平行” 即可。

题解

我们首先固定 $A'B'C'D'$ ，然后尝试构造 $ABCD$ 使 $S_{ABCD} = \frac{1}{2}S_{A'B'C'D'}$

- 1. 当 $AC \parallel B'C'$ 时，显然有 $S_{ABCD} = \frac{1}{2}S_{A'B'C'D'}$
- 2. 当 AC 与 $B'C'$ 不平行时，对任意给定的 B ， S_{ABC} 面积固定， S_{BCD} 随 D 的变化单调变化，因此仅当 $BD \parallel A'B'$ 时 $S_{ABCD} = \frac{1}{2}S_{A'B'C'D'}$



M-Fundamental Skills in Data Structures

题目大意

n 个点的有根树，每个点上有点权 a_i 为 0 或 1，要求进行两种操作：

- 链点权覆盖为 0 或 1
- 查询子树内满足 $x < y$ 且 $a_x \oplus a_y \oplus a_{lca(x,y)}$ 的点对 (x, y) 的数目



思路一

- hint 1: 使用什么思路统计答案？

思路一

- hint 1: 使用什么思路统计答案?
- answer 1: 在 lca 处统计合法点对数量, 按照点对位置分类统计。

思路一

- hint 1: 使用什么思路统计答案？
- answer 1: 在 lca 处统计合法点对数量，按照点对位置分类统计。
- hint 2: 回答询问的方式是什么？

思路一

- hint 1: 使用什么思路统计答案？
- answer 1: 在 lca 处统计合法点对数量，按照点对位置分类统计。
- hint 2: 回答询问的方式是什么？
- answer 2: 树链剖分后，在每条重链上计算 lca 在该链上的答案总和，最后求和。



思路一

- hint 3: 使用什么数据结构能够更好的处理该问题？

思路一

- hint 3: 使用什么数据结构能够更好的处理该问题？
- answer 3: 用 LCT 取代树剖，可以利用实链的分治结构计算答案。

思路一

- hint 3: 使用什么数据结构能够更好的处理该问题？
- answer 3: 用 LCT 取代树剖，可以利用实链的分治结构计算答案。
- hint 4: 如何维护操作对答案的影响？

思路一

- hint 3: 使用什么数据结构能够更好的处理该问题？
- answer 3: 用 LCT 取代树剖，可以利用实链的分治结构计算答案。
- hint 4: 如何维护操作对答案的影响？
- answer 4: 同时维护全 0 和全 1 的信息，在链覆盖的时候进行替换。

思路一

在以上思考的基础上，我们考虑如何维护实链涉及的答案和。

容易发现，点对分为两类：

- lca 与某个端点重合，或者两个端点均位于 lca 虚儿子的子树中：此时我们只需要在虚边传递的过程中直接计算即可。
- lca 其中一个端点位于实儿子的子树中，另一个端点位于虚儿子的子树中：此时我们考虑，将虚儿子处端点的信息绑定到 lca 上，另一边剩下一个实端点。对于整条实链，使用类似分治的思想，即每次考虑跨过某个节点的 lca -实端点点对的数量，最终求和。

思路一

为了处理子树询问，我们需要沿虚边向上传递子树答案的信息，并在实链上同时进行答案的求和。

LCT 本身一条实链即为一个分治结构，我们每次在 splay 中向上合并信息的时候，统计 lca 位于左子树内，实端点位于右子树内的点对数目即可。

为了处理子树询问，我们还需沿虚边向上传递子树答案的信息。实际回答时，我们只需要对子树根节点 u 进行 access 操作，将 u 下面的边全部变为虚边，并利用虚边上传递的信息计算出答案。

思路一

最后，我们使用一个比较方便实现的做法维护链覆盖，即同时维护全 0 和全 1 的各类信息，在覆盖操作的时候直接进行信息的替换。这里有一个细节是，我们可以通过两段 `access` 到根节点，但仅进行 `lca` 的后缀处理的方式来取代原本 `lct` 中先换根再 `access` 的做法，这样可以避免换根操作需要维护的额外代价。

最终做法的时间复杂度为 $O(n \log n)$ ，且 LCT 相比同一个思路的树剖做法更加好写。

思路二

思路二在实现上应该比思路一更加简单，这里仅供参考。

考虑 lca 节点处的权值情况，我们发现，若 lca 处权值为 0，实际上是统计有多少对权值相同的点位于不同子树内，若 lca 处权值为 1，则统计的是权值相反的点。

统计点对数可以使用容斥的思想，以 lca 处 0 权值为例，可以先统计出整棵子树内权值相同的点对数量，再减去每棵子树内部的点对数量。而子树和即为子树内每个点处计算出的值的和。

思路二

观察计算过程，我们发现，如果一个点和父亲具有相同的权值，则在表达式中它自己的点对数量被抵消掉了。也就是说，实际上答案的贡献仅在每一条连接两个不同权值的点的边上产生。我们只需要分别维护子树内对应权值的节点数量信息，以及每条边产生的答案贡献，即可求得答案。

思路二

考虑到每次执行链覆盖，都相当于将一部分贡献边转化为非贡献边，同时在边缘产生一些额外的贡献边。我们可以在重链上直接进行贡献清除，同时在虚边处维护可能产生的额外贡献，使用节点数量的信息进行计算。同时，还需要将节点数量信息的变化向上直接传递到根节点。

我们可以使用树剖线段树或 LCT 来维护边的贡献，同时还需要一个类似的数据结构来维护子树内不同权值的节点数量。

思路三

zky 提交的代码

提交时间: 2023-07-21 13:29:56

语言: C++(g++ 7.5.0)

代码长度: 474

运行时间: <1 ms

占用内存: 0K

运行状态: 编译错误

```
1  {"SPECIAL_TM_FH_MEMO_JSON_TAG":"SPECIAL_TM_FH_MEMO_JSON_TAG","stderr":"a.cpp:1:1: error: stray '\\350' in program\n 转化成;\n2\n3  转化成计算  $(-1)^{a_x+a_y+lca(x,y)}$  的和。\n4\n5  在  $Sp=LCAS$  上计算贡献。\n6\n7  不妨先假设一手  $x,y \neq p$ 。  $x=p$  或  $y=p$  的答案是容易计算的。\n8\n9  case1 :  $x,y$  都在轻子树。\n10\n11 case2 : 其中一个在轻子树, 一个在重子树。\n12\n13 ----\n14\n15 case1: 链修改中, 大部分都是修改重子数, 对轻子树修改次数只有  $\Theta(\log n)$  个。因此 case1 是容易维护的。\n16\n17 case2:\n18\n19 维护三个值  $a,b,c$ ,  $a$  表示  $x$ ,  $b$  表示重子树和,  $c$  表示轻子树和。\n20\n21  $a$  是覆盖。  $c$  修改是单点修改。难就难在, 对  $b$  的修改。\n22\n23 注意到一个类似 lct 均摊的结构, 就是, 每次覆盖的链上总共只有  $\Theta(\log n)$  个连续断, 因此对  $b$  的修改也是  $\log$  段等差数列加减,\n24\n25 线段树维护即可。时间负载  $\Theta(n \log n + q \log^2 n)$ 。\n26\n27 不写 /wwo
```

THANKS!

AC.NOWCODER.COM